

Pre-Impact Fall Detection on a Low-Cost Microcontroller: Cross-Dataset Generalisation, Domain Adaptation and INT8 Deployment

Md. Arif Shekh*, Md. Meherab Hossain Talukder†, Tasfia Islam Prapty‡,

G. M. Imtiaz Dinar§, Tasnia Chowdhury Toshita¶

Department of Computer Science and Engineering

IUBAT—International University of Business Agriculture and Technology

Dhaka, Bangladesh

*23103022 †23103032 ‡23103286 §23103080 ¶23103038

Corresponding author: arifshekh.k8@gmail.com

Abstract—Wearable fall detection is reported as a solved problem: within-dataset accuracy is routinely above 99%. We show that this figure survives neither a change of dataset nor the constraints of a real device. We train a 7,947-parameter separable convolutional network for pre-impact fall detection, evaluate it under a strict leave-one-dataset-out protocol across SisFall, KFall, FallAIIID and UMAFall, and deploy it as a 22.5 KB INT8 model on an ESP32 costing under BDT 500. Three results follow. First, binary F1 falls from 0.907 within-dataset to between 0.344 and 0.835 on an unseen corpus, and the collapse is strongly asymmetric between datasets that appear equivalent. Second, of three domain-adaptation methods applied in ascending order of computational cost, only the cheapest—per-window instance normalisation—improves transfer; CORAL and adversarial adaptation both make it worse. Third, where that normalisation is placed decides whether the model survives quantisation at all: as a network layer it costs 30.95 points of macro-F1 under INT8, moved into the preprocessing pipeline it costs nothing. Scored per trial on KFall as in the benchmark literature, the model reaches 0.985 sensitivity and 0.991 specificity at a mean lead time of 318 ms. We further argue that window-level specificity, the metric this literature reports, is close to meaningless for a continuously running detector, and report false alarms per hour instead.

Index Terms—fall detection, pre-impact, TinyML, cross-dataset generalisation, domain adaptation, quantisation, wearable sensors, ESP32

I. INTRODUCTION

Falls are a leading cause of injury-related death among older adults, and a wearable detector that fires *before* body–ground impact can trigger protective measures rather than merely summon help afterwards. Two gaps persist in this literature.

Untested generalisation. Within-dataset accuracy is saturated near 99%, but cross-repository benchmarking shows substantial degradation once evaluation leaves the training corpus [1], [2]. Most papers never leave it.

Unverified deployment. Nearly all reported inference runs on desktop GPUs. Work combining leave-one-dataset-out evaluation and measured microcontroller performance is rare.

This paper addresses both. Our contributions are:

- 1) A leave-one-dataset-out evaluation across four public corpora with a domain-adaptation ladder ordered by *on-device cost*, showing that the cheapest method is the only one that helps (Section V-B).
- 2) Pre-impact detection validated on video-grounded temporal labels, reported both at trial level for comparability with published work and at window level with false alarms per hour, which is what determines whether a device is wearable (Section V-C).
- 3) A deployment study on an ESP32, including a quantisation result we believe is not previously reported: the placement of instance normalisation relative to the quantisation boundary determines whether an INT8 model works at all (Section V-F).

All code, frozen preprocessing constants, the exported model and the fold-level result files are public.¹

II. RELATED WORK

Datasets and annotation. SisFall [3] provides 38 subjects at 200 Hz but labels only at trial level; Musci *et al.* [4] added per-sample three-class annotations, produced by expert panels working from sensor traces without video reference—a process subsequent authors have criticised as necessarily subjective. KFall [5] publishes fall-onset and impact frames derived from synchronised video, making it the more authoritative source for pre-impact work and the primary corpus here. FallAIIID [6] and UMAFall [7] supply independent hardware and multiple placements.

Pre-impact detection. Yu *et al.* [5] benchmark threshold, SVM and ConvLSTM models on KFall, the last reaching 99.32% sensitivity and 99.01% specificity with 403 ± 163 ms lead time. TinyFallNet [8] targets constrained devices, report-

¹<https://github.com/arifshekhk8/preimpact-fall-detection-tinyml>

ing 86.67%/97.97% and 477.7 ± 5.8 ms. PreFallKD [9] reports 551.3 ms lead time via knowledge distillation.

Cross-dataset evaluation. Silva *et al.* [1] evaluate 13 algorithms across 11 repositories and find that all degrade under cross-dataset conditions. That the gap exists is therefore established; our contribution is not to rediscover it but to ask which mitigations are affordable on a microcontroller, and to show that the answer inverts the expected ordering.

MCU deployment. Torti *et al.* [10] deploy a recurrent network to an embedded platform using the same annotations discussed above.

III. DATA AND PREPROCESSING

A. Corpora, and an audit of what they actually contain

We use four public datasets. Auditing them before modelling produced four findings that constrain what can be claimed, and we state them rather than discovering them in review.

- 1) **The SisFall Enhanced annotations are not recoverable** from the available public mirror, which ships shuffled, pre-windowed tensors without subject identifiers. We attempted correlation-based re-alignment against the original recordings; recovery peaked at 22.5% against a time-reversed null control and *fell* under harder low-pass filtering, establishing that figure as a ceiling rather than a waypoint. Contribution 2 therefore rests on KFall alone.
- 2) **The available SisFall mirror contains 25 of 38 subjects**, missing 13 of the 15 older participants—the cohort that gives the dataset its distinctive value.
- 3) **UMAFall’s waist sensor samples at 20.1 Hz**, below our 50 Hz working rate; only its pocket smartphone reaches 200 Hz, and that stream logs no gyroscope. Its fold therefore measures a combined domain, placement and modality shift.
- 4) **FallAID rests gravity on a different axis and sign** from SisFall and KFall. Left uncorrected this is indistinguishable from genuine domain shift, and a cross-dataset result would have measured an upside-down sensor rather than a distribution difference.

We exclude UP-Fall entirely: its accelerometers sample at approximately 18.4 Hz, which cannot reach 50 Hz without fabricating signal content.

B. Frozen preprocessing

Preprocessing is defined once, in a module imported by both the training scripts and the firmware generator, and is hashed into every cached artifact. Channels are the waist or low-back accelerometer and gyroscope, $(a_x, a_y, a_z, g_x, g_y, g_z)$. Signals are converted to g and deg/s using each source’s documented scaling, verified by confirming that resting magnitude is 1.0 ± 0.05 g in every corpus. All datasets are rotated onto one gravity convention using signed permutation matrices constrained to determinant +1, so gyroscope handedness is preserved. Anti-alias filtering precedes decimation to 50 Hz; slicing without filtering would alias the impact transient into the passband. Windows are 100×6 (2.0 s) at a 0.5 s stride.

TABLE I
WITHIN-DATASET BASELINE, SISFALL, POST-FALL TASK. COMPARATORS USE 5-FOLD SUBJECT-GROUPED CV; THE PROPOSED MODEL ADDITIONALLY USES FULL 25-FOLD LOSO.

Model	Params	Sens.	Spec.	Macro-F1
SMV threshold	2	0.763	0.934	0.484
SVM	—	0.958	0.997	0.977
Random Forest	—	0.952	0.997	0.975
1D-CNN	102,211	0.956	0.995	0.969
CNN-LSTM	47,811	0.954	0.996	0.972
Proposed	7,947	0.913	0.993	0.950
Proposed (LOSO)	7,947	0.882	0.993	0.953

Every split is by subject or by dataset, never by window. At 75% overlap a random window split places near-identical windows on both sides of the boundary and reports accuracy above 99% that means nothing.

IV. METHOD

The microcontroller memory budget is the design driver. Fig. 1 shows the network and the pipeline it sits in. Batch normalisation and ReLU follow each convolution, with dropout 0.3 before the head. Separable convolutions and global average pooling rather than flattening keep the total at **7,947 parameters**; a flatten at the same widths would cost 51,200 in one layer. The three-class head (background / alert / fall) yields binary metrics by merging the latter two.

Training uses Adam with cosine decay from 10^{-3} , batch 128, up to 100 epochs, early stopping on validation macro-F1 with patience 15, and class weighting rather than oversampling—at 75% overlap, oversampling duplicates near-identical windows across the split boundary.

A. Adaptation ladder

We climb three rungs in ascending order of on-device cost: **per-window instance normalisation** (two passes over 100 samples, negligible on a 240 MHz core); **CORAL** [11], aligning second-order feature statistics; and **DANN** [12], adversarial adaptation via gradient reversal. None uses target labels at any point.

V. RESULTS

A. Within-dataset performance is saturated

Table I confirms the premise: every learned model lands between 0.950 and 0.977 macro-F1. The proposed network gives up roughly two points against the best comparator for a thirteen-fold reduction in parameters against the 1D-CNN.

Table II repeats the comparison on the task that actually ships. The ordering is preserved: the proposed model trails the best comparator by roughly five points of macro-F1 while using 1/57 of Random Forest’s node count. The SMV threshold detector, at 0.326, shows how much of the pre-impact task is beyond a hand-tuned rule.

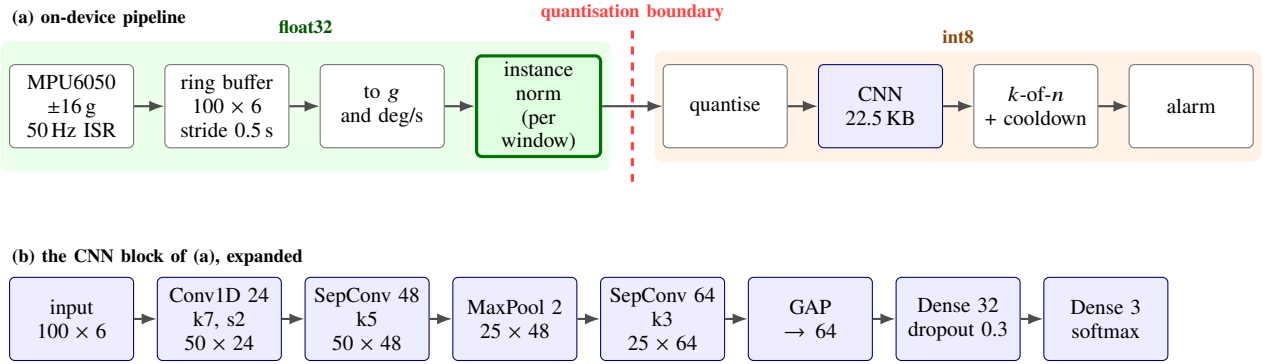


Fig. 1. The deployed system. **(a)** The on-device pipeline, shaded by numeric domain. Sampling is hardware-timer driven rather than a polled delay, because loop jitter accumulates and shifts window content away from the training distribution. Per-window instance normalisation is deliberately placed on the *float* side of the quantisation boundary: as a layer inside the graph it is mathematically identical but costs 30.95 points of macro-F1 under INT8 (Section V-F). **(b)** The network. Separable convolutions and global average pooling hold it to 7,947 parameters; batch normalisation and ReLU follow each convolution, and a flatten in place of the pooling would cost 51,200 parameters in a single layer.

TABLE II

WITHIN-DATASET *PRE-IMPACT* BASELINES ON KFall, THREE-CLASS TASK, 5-FOLD SUBJECT-GROUPED CV.

Model	Params	Sens.	Spec.	Macro-F1
Random Forest	451,336	0.922	0.994	0.920
SVM	—	0.935	0.987	0.914
1D-CNN	102,211	0.936	0.985	0.914
CNN-LSTM	47,811	0.958	0.973	0.907
Proposed	7,947	0.936	0.959	0.874
SMV threshold	2	0.737	0.911	0.326

TABLE III

LEAVE-ONE-DATASET-OUT BINARY F1. TRAIN ON TWO CORPORA, TEST ON A THIRD; NO FINE-TUNING AND NO TARGET LABELS. WITHIN-DATASET REFERENCE: 0.907.

Fold	Test	None	Inst. n.	CORAL	DANN
A	FallAllD	0.344	0.388	0.179	0.307
B	KFall	0.835	0.873	0.644	0.834
C	SisFall	0.799	0.830	0.617	0.728
D	UMAFall [†]	0.475	0.423	0.005	0.299

[†] Accelerometer-only, pocket placement: combined domain and placement shift.

B. Cross-dataset generalisation

Table III is the central result. Against the proposed model’s within-dataset binary F1 of 0.907, performance collapses on an unseen corpus, and it does so asymmetrically: KFall transfers comparatively well (0.835) while FallAllD barely transfers at all (0.344), despite both being waist-mounted at comparable rates.

The adaptation ladder inverts. Per-window instance normalisation—the cheapest rung, and the only one that fits comfortably on the target hardware—is the only method that improves transfer, gaining 0.031–0.044 F1 on folds A–C. CORAL degrades every fold, collapsing to 0.005 on UMAFall. DANN, the most expensive, is worse than no adaptation everywhere. We report this as measured; the practical reading

TABLE IV

PRE-IMPACT PERFORMANCE ON KFall, POOLED OVER 32 LOSO FOLDS, SCORED PER TRIAL AS IN [5]. PUBLISHED ROWS ARE AS REPORTED IN THE CITED WORKS.

Method	Sens.	Spec.	Lead time (ms)
Threshold [5]	0.955	0.834	333 ± 160
SVM [5]	0.998	0.949	385 ± 159
ConvLSTM [5]	0.993	0.990	403 ± 163
TinyFallNet [8]	0.867	0.980	478 ± 6
Ours ($\tau=0.5$, $k=3$)	0.985	0.991	318 ± 328
Ours ($\tau=0.7$, $k=3$)	0.978	0.995	342 ± 373

is that expensive feature alignment does not earn its cost here, and that a practitioner constrained to a single technique should choose the one that is also free.

C. Pre-impact detection

Table IV places our model against published benchmarks on the same dataset and the same trial-level protocol. We are within a point of the ConvLSTM benchmark on sensitivity, match it on specificity, and give up lead time, using 7,947 parameters. Our evaluation pools 32 leave-one-subject-out folds rather than using a single held-out split, which is the stricter protocol.

Window-level specificity is misleading. A detector deciding once per 0.5 s stride makes 7,200 decisions per hour, so 99% window-level specificity still yields 72 false alarms per hour; one per hour requires 0.99986. Scored per window our model produces 293 false alarms per hour at $\tau=0.5$. Requiring k consecutive windows suppresses these—at $k=3$ the trial-level false-positive count falls from 538 to 26 of 2,729 ADL trials—but each additional window costs one 0.5 s stride of lead time, and our mean lead of 663 ms at $k=1$ is barely longer than a single stride. Fig. 2 makes the trade explicit. This tension between lead time and false-alarm rate is, we argue, the binding constraint on deployable pre-impact detection, and it indicts the stride rather than the model.

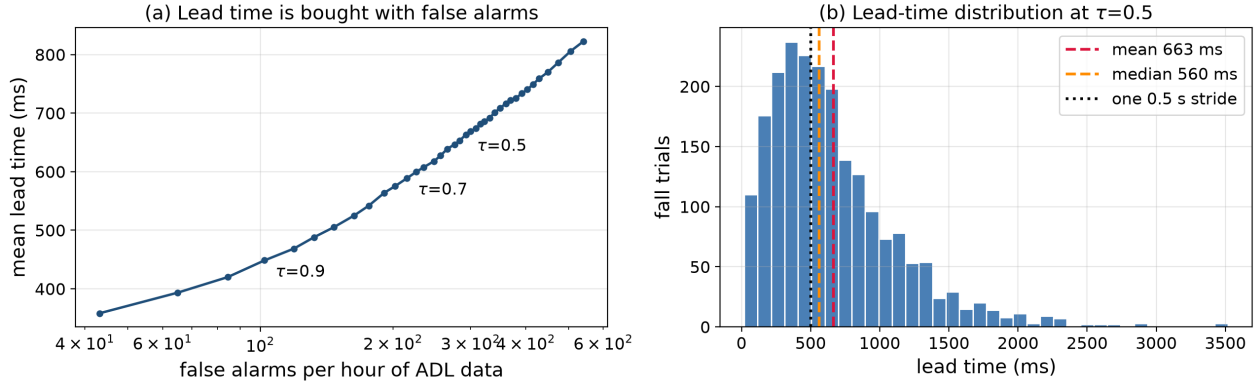


Fig. 2. The central operational trade-off, on KFall pooled over 32 LOSO folds. (a) Mean lead time against false alarms per hour of ADL data as the decision threshold is swept: every millisecond of lead time is bought with false alarms. (b) The lead-time distribution at $\tau=0.5$. The dotted line marks a single 0.5 s inference stride. Because the mean lead of 663 ms is barely longer than one stride, any k -of- n debouncing—the obvious remedy for the false-alarm rate in (a)—consumes most of the lead time it is meant to protect. The long right tail also means the mean overstates the typical case, which is why we report the distribution rather than the mean alone.

D. Ablations

Window length matters most: 2.0 s reaches 0.903 binary F1 against 0.871 at 1.5 s and 0.804 at 1.0 s. Sampling at 100 Hz buys only 0.006 F1 over 50 Hz for twice the compute and twice the input tensor, so 50 Hz is justified on evidence rather than convention. Dropping the gyroscope costs 2.6 points (0.905 to 0.880) and saves 504 parameters plus the sensor’s power draw. On FallAllD’s three mounting points, neck (0.644) outperforms waist (0.551) and wrist (0.476).

E. Prototype system

The device is an ESP32 DevKit V1 (240 MHz dual core, 320 KB usable RAM, 4 MB flash) with an MPU6050 IMU on I²C at 400 kHz, an active buzzer and a status LED, powered from a 1000 mAh LiPo cell through a TP4056 charger. The bill of materials totals BDT 1,450, of which the microcontroller is BDT 450.

The IMU is configured to match training exactly: ± 16 g (2048 LSB/g), ± 2000 dps (16.4 LSB/dps), the on-chip digital low-pass filter enabled as an analogue anti-alias stage, and a sample-rate divider of 19 giving $1000/(1+19) = 50$ Hz. The ± 16 g range is not optional: impacts routinely exceed 8 g and a clipped peak destroys exactly the signal the model depends on.

A rule-based two-stage detector—free-fall below 0.65 g followed by impact above 2.50 g or rotation above 250 deg/s within 1.5 s—runs on the same live samples, giving an on-device instance of the SMV threshold comparator of Table I and allowing both detectors to be compared against identical physical motion.

F. Deployment and quantisation

Table V summarises the deployed model. Quantisation to INT8 reduces it from 163.3 KB to 22.5 KB, a factor of 7.3, while macro-F1 on the held-out subjects is unchanged to within a third of a point and the two versions agree on 99.4% of windows.

TABLE V
DEPLOYED MODEL. TRAINED ON SisFALL, KFall AND FallAllD; EVALUATED ON HELD-OUT SUBJECTS. SIZES AND ACCURACIES ARE MEASURED; ON-DEVICE TIMING IS REPORTED IN THE TEXT.

Metric	FP32	INT8
Parameters	7,947	
Model size (KB)	163.3	22.5
Tensor arena (KB, est.)	—	8.0
Macro-F1 (held out)	0.711	0.714
Sensitivity	0.922	0.922
Specificity	0.894	0.893
FP32/INT8 agreement	0.994	

Where normalisation lives decides whether quantisation works. Instance normalisation is mathematically identical whether applied as a network layer or in preprocessing. Under full-integer quantisation it is not, because per-window mean and variance produce intermediate tensors whose dynamic range a single per-tensor scale cannot represent:

Instance norm	FP32	INT8	Agreement
In graph	0.668	0.359	0.686
In pipeline	0.701	0.704	0.994

Moving the operation across the boundary marked in Fig. 1 changes a 30.95-point collapse into no loss at all. The firmware performs the same two-pass computation in float before quantising. We report this because the failure is silent: the FP32 model validates normally, and only the deployed INT8 model misbehaves.

The deployed model excludes UMAFall from training, because its 200 Hz stream carries no gyroscope and training on identically zero channels would teach the network that a dead sensor is normal. On held-out subjects spanning the three remaining corpora (814 fall and 1,136 ADL trials) it reaches 0.958 trial-level sensitivity and 0.836 specificity at a mean lead time of 539 ± 509 ms, with 68% of detections preceding

impact. Requiring two consecutive windows raises specificity to 0.975 but reduces the pre-impact fraction to 16%—the same tension quantified in Fig. 2, now visible in the deployed system rather than in simulation. These figures are lower than Table IV because that model is trained and tested within KFall alone, whereas the deployed model is held to three heterogeneous corpora at once.

Status of the hardware evaluation. The prototype is assembled and the firmware runs the quantised model on the device, with sampling, normalisation, inference and the alarm path implemented as described in Section V-E. The model occupies 22.5 KB of flash against the 4 MB available, and the tensor arena is estimated at 8.0 KB from the converted model’s tensors, well inside the ESP32’s 320 KB of usable RAM, so the design fits the platform with substantial margin. Measured inference latency, the tensor-arena high-water mark reported by TFLite Micro at run time, and battery life under continuous inference are instrumented in the firmware but are not yet collected over a sufficient number of trials to report, and we therefore omit them rather than estimate them. Completing that measurement, and a free-living false-alarm study, is the immediate next step.

VI. DISCUSSION AND LIMITATIONS

The cross-dataset gap is not uniform, and its asymmetry is more informative than its magnitude. FallAIID resists transfer despite nominally matching the training sources on placement, which suggests that sensor hardware and recording protocol dominate placement as sources of shift.

Limitations, stated plainly: falls are simulated by mostly young participants rather than real falls by older adults; the available SisFall mirror omits most of its older cohort; annotations in every source except KFall were produced without video reference; fold D confounds domain shift with placement and modality; the false-alarm rates reported here come from laboratory ADL recordings rather than continuous free-living wear; and on-device inference latency, run-time memory and battery life are instrumented in the firmware but not yet measured over enough trials to report.

VII. CONCLUSION

A 7,947-parameter network quantised to 22.5 KB attains pre-impact performance comparable to a substantially larger published benchmark, and runs on a microcontroller costing under BDT 500. Its within-dataset accuracy does not survive a change of dataset, and of three adaptation methods only the cheapest helps. The tension between lead time and false-alarm rate, rather than raw accuracy, is what stands between this class of model and a deployable device; shortening the inference stride is the most promising route and is our immediate future work.

REFERENCES

[1] J. Silva, E. Casilari, and R. García-Bermúdez, “Cross-dataset evaluation of wearable fall detection systems using data from real falls and long-term monitoring of daily life,” *Measurement*, vol. 234, p. 114843, 2024.

[2] E. Casilari, J. A. Santoyo-Ramón, and J. M. Cano-García, “Analysis of public datasets for wearable fall detection systems,” *Sensors*, vol. 17, no. 7, p. 1513, 2017.

[3] A. Sucerquia, J. D. López, and J. F. Vargas-Bonilla, “SisFall: A fall and movement dataset,” *Sensors*, vol. 17, no. 1, p. 198, 2017.

[4] M. Musci, D. De Martini, N. Blago, T. Facchinetti, and M. Piastra, “Online fall detection using recurrent neural networks on smart wearable devices,” *IEEE Transactions on Emerging Topics in Computing*, vol. 9, no. 3, pp. 1276–1289, 2020.

[5] X. Yu, J. Jang, and S. Xiong, “A large-scale open motion dataset (KFall) and benchmark algorithms for detecting pre-impact fall of the elderly using wearable inertial sensors,” *Frontiers in Aging Neuroscience*, vol. 13, p. 692865, 2021.

[6] M. Saleh, M. Abbas, and R. Le Bouquin Jeannès, “FallAIID: An open dataset of human falls and activities of daily living for classical and deep learning applications,” *IEEE Sensors Journal*, vol. 21, no. 2, pp. 1849–1858, 2021.

[7] E. Casilari, J. A. Santoyo-Ramón, and J. M. Cano-García, “UMAFall: A multisensor dataset for the research on automatic fall detection,” *Procedia Computer Science*, vol. 110, pp. 32–39, 2017.

[8] B. Kim, S.-W. Kim, and S. Lee, “TinyFallNet: A lightweight pre-impact fall detection model,” *Sensors*, vol. 23, no. 20, p. 8459, 2023.

[9] T.-H. Wu, C.-H. Chao, C.-Y. Lin, and Y.-C. Tseng, “PreFallKD: Pre-impact fall detection via CNN-ViT knowledge distillation,” in *arXiv preprint arXiv:2303.03634*, 2023.

[10] E. Torti, A. Fontanella, M. Musci, N. Blago, D. Pau, F. Leporati, and M. Piastra, “Embedding recurrent neural networks in wearable systems for real-time fall detection,” *Microprocessors and Microsystems*, vol. 71, p. 102895, 2019.

[11] B. Sun, J. Feng, and K. Saenko, “Return of frustratingly easy domain adaptation,” *Proc. AAAI Conference on Artificial Intelligence*, 2016.

[12] Y. Ganin, E. Ustinova, H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, M. Marchand, and V. Lempitsky, “Domain-adversarial training of neural networks,” *Journal of Machine Learning Research*, vol. 17, no. 59, pp. 1–35, 2016.